

Коробейников Артём Юрьевич Экзамен

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.linear_model import LogisticRegression
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten, Dense,
Dropout
from tensorflow.keras.utils import to_categorical
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
import numpy as np
from PIL import Image, ImageDraw, ImageFont
import matplotlib.pyplot as plt
import random
import os

def exam()::
    import numpy as np
    from PIL import Image, ImageDraw, ImageFont
    import matplotlib.pyplot as plt
    import random
    import os
    import seaborn as sns # Для красивой матрицы ошибок

    from sklearn.model_selection import train_test_split
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.svm import SVC
    from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

    # --- Конфигурация ---
    image_size = (16, 16) # Ширина, Высота изображения в пикселях
    chars = ['A', 'B', 'C', 'D', 'E', 'F'] # Буквы для генерации
```

```

num_images_per_char = 100 # Количество синтетических изображений для
каждой буквы
font_size = 12 # Размер шрифта, регулируйте для лучшей читаемости на
16x16
num_viz_samples_per_char = 5 # Количество примеров для визуализации на
каждую букву
test_size_ratio = 0.20 # 20% данных для тестовой выборки
random_seed = 42 # Для воспроизводимости результатов

# --- Загрузка шрифта ---
font = None
try:
    font_paths = [
        "arial.ttf", # Windows
        "/usr/share/fonts/truetype/freefont/FreeSans.ttf", #
Debian/Ubuntu
        "/Library/Fonts/Arial.ttf", # macOS common path
        "/System/Library/Fonts/Supplemental/Arial.ttf", # macOS newer
path
        "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf", # Другой
распространенный шрифт Linux
        os.path.join(os.path.dirname(__file__), "Arial.ttf") # Если
Arial.ttf находится в той же директории
    ]
    for path in font_paths:
        if os.path.exists(path):
            font = ImageFont.truetype(path, font_size)
            print(f"Используется шрифт: {path}")
            break
    if font is None:
        font = ImageFont.load_default()
        print(
            "Внимание: Не удалось найти системный шрифт. Используется
стандартный шрифт PIL. Качество может быть низким.")
    except Exception as e:
        print(f"Ошибка при загрузке шрифта: {e}. Используется стандартный
шрифт PIL.")
        font = ImageFont.load_default()

# --- Часть А: Создание набора данных ---

def generate_letter_image(letter, image_size, font):
    """
    Генерирует серое изображение заданной буквы.
    """
    img = Image.new('L', image_size, color=0)
    draw = ImageDraw.Draw(img)

```

```

try:
    if hasattr(draw, 'textbbox'):
        bbox = draw.textbbox((0, 0), letter, font=font)
        text_width = bbox[2] - bbox[0]
        text_height = bbox[3] - bbox[1]
        x = (image_size[0] - text_width) / 2 - bbox[0]
        y = (image_size[1] - text_height) / 2 - bbox[1]
    else:
        text_width, text_height = draw.textsize(letter, font=font)
        x = (image_size[0] - text_width) / 2
        y = (image_size[1] - text_height) / 2
        if font == ImageFont.load_default():
            y += 1
except Exception as e:
    text_width, text_height = font.getsize(letter) if hasattr(font,
'getsize') else (font_size, font_size)
    x = (image_size[0] - text_width) / 2
    y = (image_size[1] - text_height) / 2

draw.text((x, y), letter, font=font, fill=255)
return np.array(img)

print("\n--- Часть А: Создание набора данных ---")
all_images = []
all_labels = []

for char in chars:
    print(f"Генерируется {num_images_per_char} изображений для символа '{char}'...")
    for _ in range(num_images_per_char):
        img_array = generate_letter_image(char, image_size, font)
        all_images.append(img_array)
        all_labels.append(char)

print(
    f"Набор данных сгенерирован: {len(all_images)} изображений, каждое
размером {image_size[0]}x{image_size[1]} пикселей.")

X = np.array(all_images)
y = np.array(all_labels)

# --- Часть А: Визуализация примеров ---
print("\n--- Часть А: Визуализация примеров ---")
fig, axes = plt.subplots(nrows=len(chars), ncols=num_viz_samples_per_char,
                        figsize=(num_viz_samples_per_char * 1.5,
len(chars) * 1.5))
fig.suptitle(f"Синтетические изображения букв
({image_size[0]}x{image_size[1]} пикселей)", fontsize=16)

```

```

if len(chars) == 1:
    axes = np.array([axes])

for i, char in enumerate(chars):
    char_indices = np.where(y == char)[0]

    if len(char_indices) >= num_viz_samples_per_char:
        selected_indices = random.sample(list(char_indices),
num_viz_samples_per_char)
    else:
        selected_indices = char_indices
        print(f"Внимание: Доступно только {len(char_indices)} изображений
для '{char}', отображаются все.")

    for j, idx in enumerate(selected_indices):
        ax = axes[i, j]
        ax.imshow(X[idx], cmap='gray')
        ax.set_title(f"Класс: {y[idx]}", fontsize=8)
        ax.axis('off')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()

# --- Предобработка данных (общая для Частей В и С) ---
print("\n--- Предобработка данных для классификации ---")
X_flat = X.reshape(X.shape[0], -1)
print(f"Размерность данных после преобразования в одномерный массив:
{X_flat.shape}")

X_train, X_test, y_train, y_test = train_test_split(
    X_flat, y, test_size=test_size_ratio, random_state=random_seed,
stratify=y
)
print(f"Размер обучающей выборки: {X_train.shape[0]} изображений")
print(f"Размер тестовой выборки: {X_test.shape[0]} изображений")

# --- Часть В: Случайный лес ---
print("\n--- Часть В: Случайный лес ---")
print("2. Обучение и оценка случайного леса")
print("Обучение RandomForestClassifier...")
random_forest_model = RandomForestClassifier(n_estimators=100,
random_state=random_seed, n_jobs=-1, verbose=0)
random_forest_model.fit(X_train, y_train)
print("Обучение завершено.")

y_pred_rf = random_forest_model.predict(X_test)
accuracy_rf = accuracy_score(y_test, y_pred_rf)

```

```

    print(f"\nТочность классификатора Случайный лес на тестовой выборке:
{accuracy_rf:.4f}")

cm_rf = confusion_matrix(y_test, y_pred_rf, labels=chars)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_rf, annot=True, fmt='d', cmap='Blues',
            xticklabels=chars, yticklabels=chars)
plt.title('Матрица ошибок для RandomForestClassifier')
plt.xlabel('Предсказанный класс')
plt.ylabel('Истинный класс')
plt.show()

print("\nОтчет о классификации (RandomForestClassifier):")
print(classification_report(y_test, y_pred_rf, target_names=chars))

# --- Часть С: Опорные векторы (SVM) ---
print("\n--- Часть С: Опорные векторы (SVM) ---")
print("2. Обучение и оценка SVM")

# Модель SVM с линейным ядром
print("\nОбучение SVM с линейным ядром...")
svm_linear_model = SVC(kernel='linear', random_state=random_seed,
verbose=False)
svm_linear_model.fit(X_train, y_train)
print("Обучение SVM с линейным ядром завершено.")

y_pred_svm_linear = svm_linear_model.predict(X_test)
accuracy_svm_linear = accuracy_score(y_test, y_pred_svm_linear)
print(f"Точность SVM с линейным ядром на тестовой выборке:
{accuracy_svm_linear:.4f}")

cm_svm_linear = confusion_matrix(y_test, y_pred_svm_linear, labels=chars)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_svm_linear, annot=True, fmt='d', cmap='Greens',
            xticklabels=chars, yticklabels=chars)
plt.title('Матрица ошибок для SVM (Linear Kernel)')
plt.xlabel('Предсказанный класс')
plt.ylabel('Истинный класс')
plt.show()

print("\nОтчет о классификации (SVM, Linear Kernel):")
print(classification_report(y_test, y_pred_svm_linear,
target_names=chars))

# Модель SVM с радиальным базисным ядром (RBF)
print("\nОбучение SVM с радиальным базисным ядром (RBF)...")
# Параметры C и gamma могут быть настроены для лучшей производительности
svm_rbf_model = SVC(kernel='rbf', random_state=random_seed, verbose=False)

```

```

svm_rbf_model.fit(X_train, y_train)
print("Обучение SVM с RBF ядром завершено.")

y_pred_svm_rbf = svm_rbf_model.predict(X_test)
accuracy_svm_rbf = accuracy_score(y_test, y_pred_svm_rbf)
print(f"Точность SVM с RBF ядром на тестовой выборке:
{accuracy_svm_rbf:.4f}")

cm_svm_rbf = confusion_matrix(y_test, y_pred_svm_rbf, labels=chars)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_svm_rbf, annot=True, fmt='d', cmap='Purples',
            xticklabels=chars, yticklabels=chars)
plt.title('Матрица ошибок для SVM (RBF Kernel)')
plt.xlabel('Предсказанный класс')
plt.ylabel('Истинный класс')
plt.show()

print("\nОтчет о классификации (SVM, RBF Kernel):")
print(classification_report(y_test, y_pred_svm_rbf, target_names=chars))

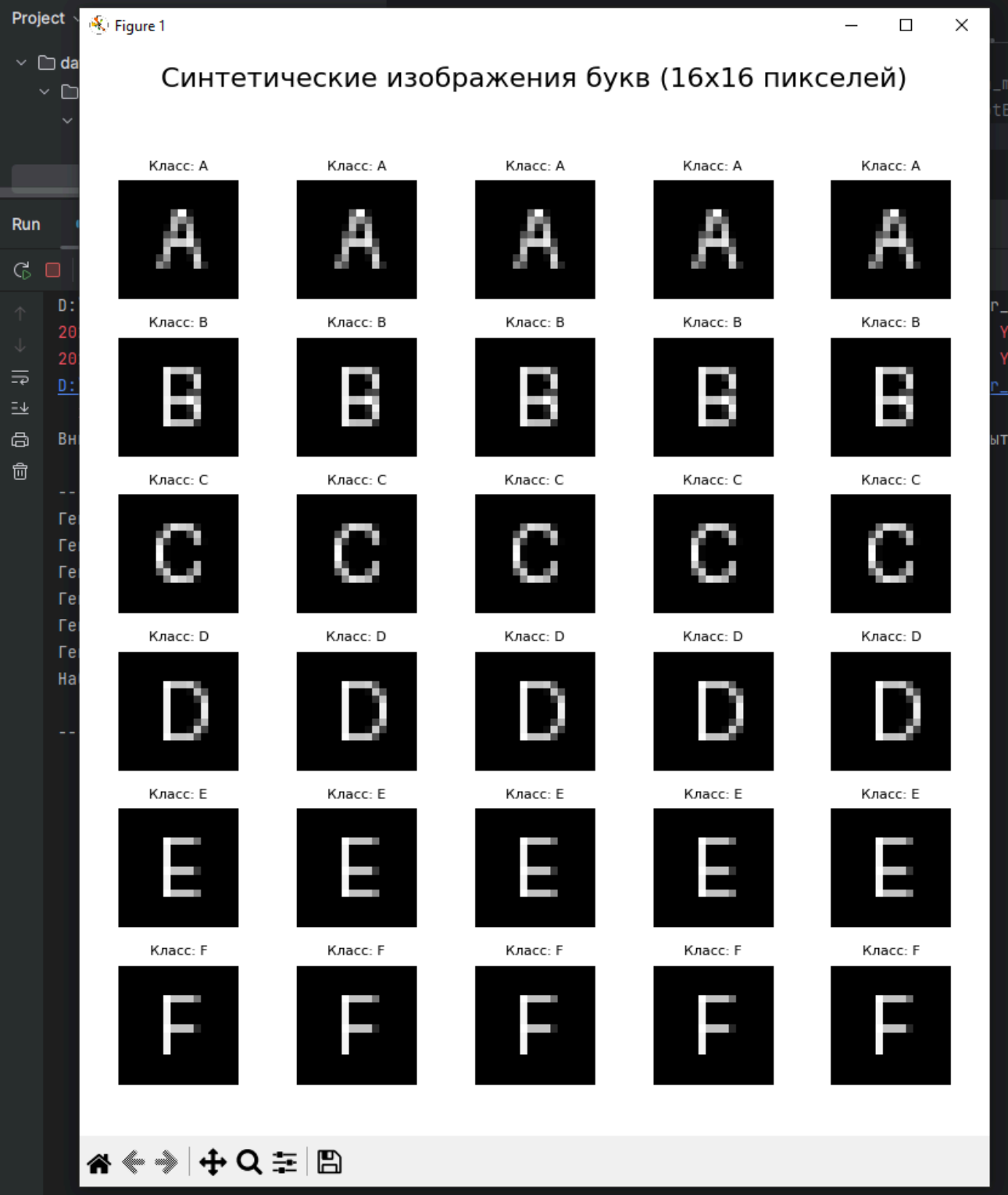
# Сравнение точности ядер
print("\n--- Сравнение точности классификаторов ---")
print(f"Точность Случайного леса: {accuracy_rf:.4f}")
print(f"Точность SVM (Linear Kernel): {accuracy_svm_linear:.4f}")
print(f"Точность SVM (RBF Kernel): {accuracy_svm_rbf:.4f}")

print("\nСкрипт завершен для Частей А, В и С.")

if __name__ == '__main__':
    exam()

```

Вывод



DS data_analysys_and_aritificial_intelligence_tasks_2_year_of_study

main

exam

Project

task2_1.py task3_1.py task4_1.py exam.py

data_analysys_and_aritificial_intelligence_tasks_2_year_of_study

tasks_third_semester

exam

venv library root

gitignore

Run

exam

D:\python_programming\for_git_repositories\data_analysys_and_aritificial_intelligence_tasks_2_year_of_study\exam.py

2025-12-24 20:00:15.054407: I tensorflow/core/util/port.cc:153] oneDNN custom op...

2025-12-24 20:00:16.336439: I tensorflow/core/util/port.cc:153] oneDNN custom op...

D:\python_programming\for_git_repositories\data_analysys_and_aritificial_intelligence_tasks_2_year_of_study\exam.py

if not hasattr(np, "object"):

Внимание: Не удалось найти системный шрифт. Используется стандартный шрифт PIL. К...

--- Часть A: Создание набора данных ---

Генерируется 100 изображений для символа 'A'...

Генерируется 100 изображений для символа 'B'...

Генерируется 100 изображений для символа 'C'...

Генерируется 100 изображений для символа 'D'...

Генерируется 100 изображений для символа 'E'...

Генерируется 100 изображений для символа 'F'...

Набор данных сгенерирован: 600 изображений, каждое размером 16x16 пикселей.

--- Часть A: Визуализация примеров ---

--- Предобработка данных для классификации ---

Размерность данных после преобразования в одномерный массив: (600, 256)

Размер обучающей выборки: 480 изображений

Размер тестовой выборки: 120 изображений

--- Часть B: Случайный лес ---

2. Обучение и оценка случайного леса

Обучение RandomForestClassifier...

Обучение завершено.

Точность классификатора Случайный лес на тестовой выборке: 1.0000

Figure 1

Матрица ошибок для RandomForestClassifier

A	20	0	0	0	0	0
B	0	20	0	0	0	0
C	0	0	20	0	0	0
D	0	0	0	20	0	0
E	0	0	0	0	20	0
F	0	0	0	0	0	20
	A	B	C	D	E	F

Истинный класс

Предсказанный класс

20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0

Project

data_analysys_and_aritificialIntelligence_ta

tasks_third_semester

exam

.venv library root

.gitignore

task2_1.py

task3_1.py

task4_1.py

exam.py

18

19

20

21

22

23

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

from sklearn.preprocessing import LabelEncoder, OneHotEncoder

import numpy as np

from PIL import Image, ImageDraw, ImageFont

import matplotlib.pyplot as plt

Run

exam

--- Предобработка данных для классификации ---

Размерность данных после преобразования в одномерный массив: (600, 256)

Размер обучающей выборки: 480 изображений

Размер тестовой выборки: 120 изображений

--- Часть B: Случайный лес ---

2. Обучение и оценка случайного леса

Обучение RandomForestClassifier...

Обучение завершено.

Точность классификатора Случайный лес на тестовой выборке: 1.0000

Отчет о классификации (RandomForestClassifier):

	precision	recall	f1-score	support
A	1.00	1.00	1.00	20
B	1.00	1.00	1.00	20
C	1.00	1.00	1.00	20
D	1.00	1.00	1.00	20
E	1.00	1.00	1.00	20
F	1.00	1.00	1.00	20

accuracy

macro avg

weighted avg

1.00

1.00

1.00

1.00

1.00

1.00

120

120

120

--- Часть C: Опорные векторы (SVM) ---

2. Обучение и оценка SVM

Обучение SVM с линейным ядром...

Обучение SVM с линейным ядром завершено.

Точность SVM с линейным ядром на тестовой выборке: 1.0000

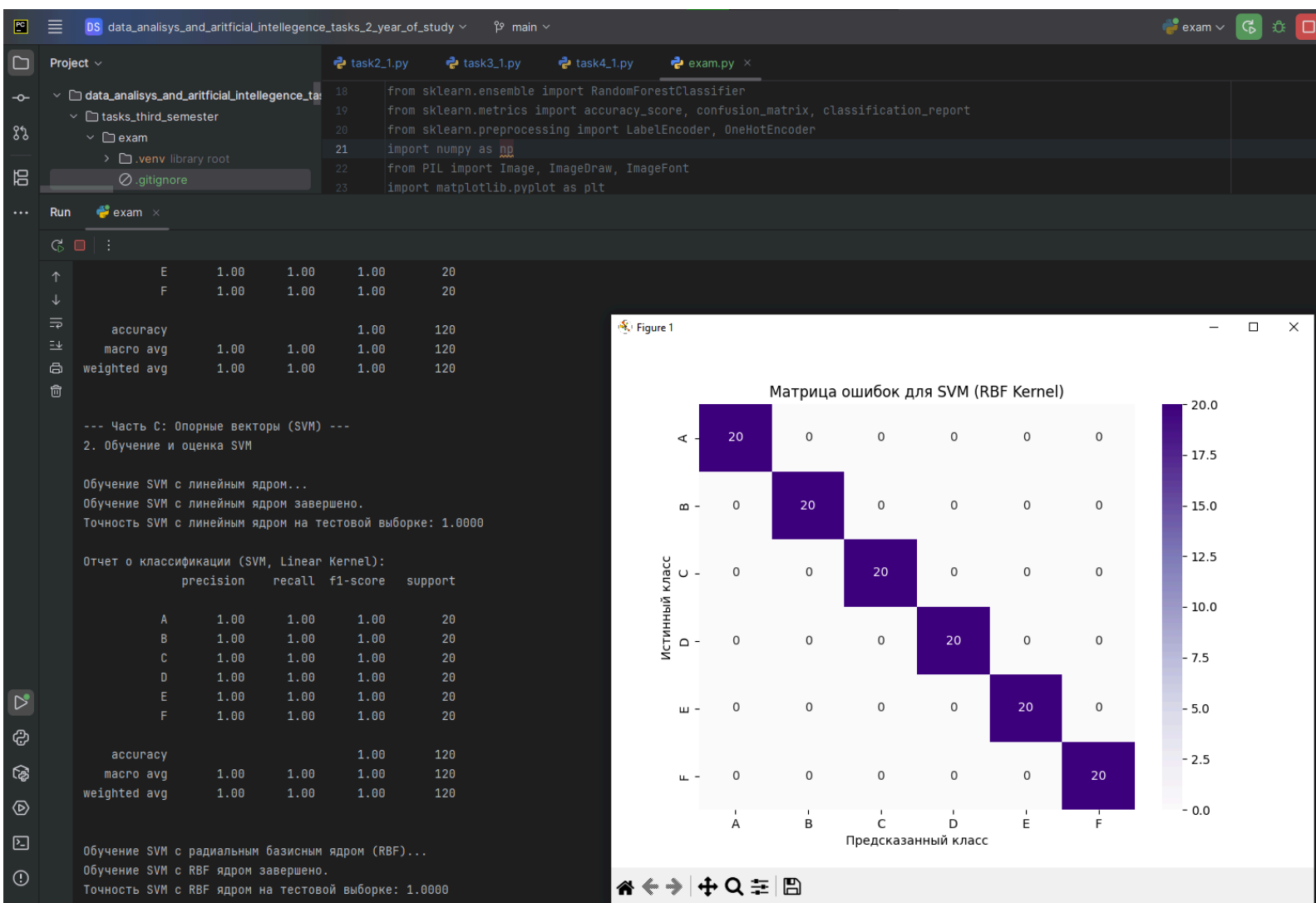
Figure 1

Матрица ошибок для SVM (Linear Kernel)

	A	B	C	D	E	F
A	20	0	0	0	0	0
B	0	20	0	0	0	0
C	0	0	20	0	0	0
D	0	0	0	20	0	0
E	0	0	0	0	20	0
F	0	0	0	0	0	20

Истинный класс

Предсказанный класс



Вывод в консоли

D:\python_programming\for_git_repositories\data_analysys_and_aritificial_intelligence_tasks_2_year_of_study\tasks_third_semester\exam\venv\Scripts\python.exe

D:\python_programming\for_git_repositories\data_analysys_and_aritificial_intelligence_tasks_2_year_of_study\tasks_third_semester\exam\exam.py

2025-12-24 19:57:28.369631: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

2025-12-24 19:57:29.699619: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

D:\python_programming\for_git_repositories\data_analysys_and_aritificial_intelligence_tasks_2_year_of_study\tasks_third_semester\exam\venv\Lib\site-packages\keras\src\export\tf2onnx_lib.py:8: FutureWarning: In the future `np.object` will be defined as the corresponding NumPy scalar.

if not hasattr(np, "object"):

Внимание: Не удалось найти системный шрифт. Используется стандартный шрифт PIL. Качество может быть низким.

--- Часть A: Создание набора данных ---

Генерируется 100 изображений для символа 'A'...

Генерируется 100 изображений для символа 'B'...

Генерируется 100 изображений для символа 'C'...

Генерируется 100 изображений для символа 'D'...

Генерируется 100 изображений для символа 'E'...

Генерируется 100 изображений для символа 'F'...

Набор данных сгенерирован: 600 изображений, каждое размером 16x16 пикселей.

--- Часть A: Визуализация примеров ---

--- Предобработка данных для классификации ---

Размерность данных после преобразования в одномерный массив: (600, 256)

Размер обучающей выборки: 480 изображений

Размер тестовой выборки: 120 изображений

--- Часть B: Случайный лес ---

2. Обучение и оценка случайного леса

Обучение RandomForestClassifier...

Обучение завершено.

Точность классификатора Случайный лес на тестовой выборке: 1.0000

Отчет о классификации (RandomForestClassifier):

	precision	recall	f1-score	support
A	1.00	1.00	1.00	20
B	1.00	1.00	1.00	20
C	1.00	1.00	1.00	20
D	1.00	1.00	1.00	20
E	1.00	1.00	1.00	20
F	1.00	1.00	1.00	20
accuracy		1.00		120
macro avg	1.00	1.00	1.00	120
weighted avg	1.00	1.00	1.00	120

--- Часть C: Опорные векторы (SVM) ---

2. Обучение и оценка SVM

Обучение SVM с линейным ядром...

Обучение SVM с линейным ядром завершено.

Точность SVM с линейным ядром на тестовой выборке: 1.0000

Отчет о классификации (SVM, Linear Kernel):

	precision	recall	f1-score	support
A	1.00	1.00	1.00	20
B	1.00	1.00	1.00	20
C	1.00	1.00	1.00	20
D	1.00	1.00	1.00	20
E	1.00	1.00	1.00	20
F	1.00	1.00	1.00	20
accuracy			1.00	120
macro avg	1.00	1.00	1.00	120
weighted avg	1.00	1.00	1.00	120

Обучение SVM с радиальным базисным ядром (RBF)...

Обучение SVM с RBF ядром завершено.

Точность SVM с RBF ядром на тестовой выборке: 1.0000

Отчет о классификации (SVM, RBF Kernel):

	precision	recall	f1-score	support
A	1.00	1.00	1.00	20
B	1.00	1.00	1.00	20
C	1.00	1.00	1.00	20
D	1.00	1.00	1.00	20
E	1.00	1.00	1.00	20
F	1.00	1.00	1.00	20
accuracy			1.00	120
macro avg	1.00	1.00	1.00	120
weighted avg	1.00	1.00	1.00	120

--- Сравнение точности классификаторов ---

Точность Случайного леса: 1.0000

Точность SVM (Linear Kernel): 1.0000

Точность SVM (RBF Kernel): 1.0000

Скрипт завершен для Частей A, B и C.

Process finished with exit code 0